

PROTOCOLE ARP

Question 1 : Signification de l'acronyme ARP, utilité et fonctionnement

ARP signifie Address Resolution Protocol.

Utilité :

Le protocole ARP permet d'obtenir l'adresse MAC (Physique / Ethernet) associée à une adresse IP.

Une machine qui connaît l'adresse IP d'une autre machine utilise ARP pour découvrir son adresse MAC.

Fonctionnement :

1. Requête ARP (Broadcast) :
La machine source envoie une trame en broadcast "who-has @IP ?" à toutes les machines du réseau local.
2. Réponse ARP (Unicast) :
La machine possédant l'adresse IP demandée répond uniquement à la machine source avec "@IP is-at @MAC".

Question 2 : Cache ARP sur Ubuntu Desktop

Commande utilisée :

`ip neighbour`

ou

`ip -4 n`

Résultat exemple :

192.168.1.1 dev enp0s3 lladdr 08:00:27:a1:b2:c3 REACHABLE

192.168.1.10 dev enp0s3 lladdr 08:00:27:d4:e5:f6 STALE

Signification de chaque ligne :

- 192.168.1.1 : Adresse IP de la machine distante
- dev enp0s3 : Interface réseau locale utilisée
- lladdr 08:00:27:a1:b2:c3 : Adresse MAC associée à cette IP
- REACHABLE : Entrée valide et récemment utilisée
- STALE : Entrée ancienne mais non vérifiée

Question 3 : Cache ARP sur Windows

Commande utilisée :

`arp -a`

Résultat exemple :

Interface : 192.168.1.100

Adresse Internet Adresse physique Type

192.168.1.1 08-00-27-a1-b2-c3 dynamique

192.168.1.255 ff-ff-ff-ff-ff-ff statique

Signification de chaque ligne :

- Interface : Adresse IP de l'interface locale
- Adresse Internet : Adresse IP de la machine distante
- Adresse physique : Adresse MAC correspondante
- Type dynamique : Entrée créée automatiquement par ARP
- Type statique : Entrée configurée manuellement ou réservée

Question 4 : Vider le cache ARP sur Ubuntu

Commande utilisée :

```
sudo ip neighbour flush dev enp0s3
```

Note :

Cette commande doit être lancée plusieurs fois car l'association avec la passerelle revient rapidement.

Pour vérifier le nom de l'interface :

```
ip a
```

Question 5 : Vider le cache ARP sur Windows

Commande utilisée :

```
arp -d *
```

ANALYSE DE TRAMES ARP ET ICMP : ping entre Ubuntu et RTBox2

Question 1 : Capture et filtre Wireshark

Procédure :

1. Lancer Wireshark sur RTBox2
2. Vider le cache ARP : `sudo ip n f dev enp0s3`
3. Lancer un ping : `ping -c 1 <IP_RTBox2>`
4. Arrêter la capture Wireshark

Filtre Wireshark utilisé :

`(arp and eth.addr == <MAC_Ubuntu>) or (icmp and ip.addr == <IP_Ubuntu>)`

Explication du filtre :

- arp and eth.addr == <MAC_Ubuntu> : trames ARP impliquant la MAC Ubuntu
- icmp and ip.addr == <IP_Ubuntu> : paquets ICMP impliquant l'IP Ubuntu
- or : permet d'afficher les 4 trames nécessaires

Les 4 trames capturées :

1. ARP Request (broadcast) : who-has @IP_RTBox2 ?
2. ARP Reply (unicast) : @IP_RTBox2 is-at @MAC_RTBox2
3. ICMP Echo Request (ping)
4. ICMP Echo Reply (pong)

Question 2 : Encapsulation des 4 trames

Trame 1 - ARP Request :

Ethernet Header

MAC Source : <MAC_Ubuntu>

MAC Destination : ff:ff:ff:ff:ff:ff

Type : 0x0806

ARP Packet

Operation : Request

Sender MAC : <MAC_Ubuntu>

Sender IP : <IP_Ubuntu>

Target MAC : 00:00:00:00:00:00

Target IP : <IP_RTBox2>

Trame 2 - ARP Reply :

Ethernet Header

MAC Source : <MAC_RTBox2>

MAC Destination : <MAC_Ubuntu>

Type : 0x0806

ARP Packet

Operation : Reply

Sender MAC : <MAC_RTBox2>

Sender IP : <IP_RTBox2>

Target MAC : <MAC_Ubuntu>

Target IP : <IP_Ubuntu>

Trame 3 - ICMP Echo Request :

Ethernet Header

MAC Source : <MAC_Ubuntu>

MAC Destination : <MAC_RTBox2>

Type : 0x0800

IP Header

IP Source : <IP_Ubuntu>

IP Destination : <IP_RTBox2>

Protocol : ICMP

ICMP Packet

Type : 8

Code : 0

Trame 4 - ICMP Echo Reply :

Ethernet Header

MAC Source : <MAC_RTBox2>

MAC Destination : <MAC_Ubuntu>

Type : 0x0800

IP Header

IP Source : <IP_RTBox2>

IP Destination : <IP_Ubuntu>

Protocol : ICMP

ICMP Packet

Type : 0

Code : 0

Question 3 : Pourquoi MAC destination = IP destination ?

Dans l'ICMP Echo Request, l'adresse MAC de destination et l'adresse IP de destination correspondent à la même machine car le prochain saut est aussi la destination finale.

MAC destination = MAC RTBox2

IP destination = IP RTBox2

Ubuntu ping directement sa passerelle RTBox2.

ANALYSE DE TRAMES ARP ET ICMP : ping entre Ubuntu et iut-rt (172.31.25.9)

Question 1 : Vérification du chemin avec traceroute

Commande utilisée :

traceroute 172.31.25.9

Résultat attendu :

1 <IP_RTBox2>

2 172.31.16.1

3 172.31.25.9

Question 2 : Capture et analyse

Observation importante :

Sans NAT, les adresses IP source et destination restent identiques tout au long du chemin.
Les adresses MAC changent à chaque saut.

NAT :

Méthode permettant à une passerelle de modifier une ou plusieurs adresses IP d'un paquet.

Question 3 : Même adresse MAC de destination

Commandes testées :

ping 172.31.16.1

ping 172.31.25.9

Observation :

L'adresse MAC de destination est la même dans les deux cas : celle de RTBox2.

Explication :

Le prochain saut est toujours RTBox2, même si les adresses IP finales sont différentes.

RÉSUMÉ POUR LE CONTRÔLE

- Voir le cache ARP :
ip neigh (Linux)
arp -a (Windows)
- Vider le cache ARP :
sudo ip neigh flush all
arp -d *
- ARP :
Requête = Broadcast
Réponse = Unicast
- MITM :
Empoisonnement du cache ARP
Preuve : la passerelle pointe vers la MAC de l'attaquant
- IP :
Savoir repérer TTL et Protocol dans Wireshark

RÉSUMÉ DES COMMANDES IMPORTANTES – TP ARP / ICMP

1. `ip neighbour`
2. Affiche le cache ARP sous Linux en montrant les associations entre adresses IP et adresses MAC.
3. `ip -4 n`

Version courte de la commande précédente, limitée aux entrées IPv4 du cache ARP.

4. `ip a`

Permet d'afficher toutes les interfaces réseau de la machine afin d'identifier le nom de l'interface utilisée (ex : `enp0s3`).

5. `sudo ip neighbour flush dev enp0s3`

Vide le cache ARP de l'interface réseau spécifiée pour forcer l'envoi de nouvelles requêtes ARP.

6. `arp -a`

Affiche le cache ARP sous Windows avec les correspondances IP / MAC connues par la machine.

7. `arp -d *`

Supprime toutes les entrées du cache ARP sous Windows.

8. `ping -c 1`

Envoie une seule requête ICMP pour tester la connectivité réseau avec une machine distante.

9. `ping`

Envoie des requêtes ICMP continues pour vérifier la communication et mesurer les temps de réponse.

10. `tracert`

Affiche le chemin suivi par les paquets IP en listant les routeurs traversés jusqu'à la destination.

11. Wireshark (filtre)

`(arp and eth.addr == <MAC_Ubuntu>) or (icmp and ip.addr == <IP_Ubuntu>)`

Permet de n'afficher que les trames ARP et ICMP liées à la machine Ubuntu lors de la capture.